

Machine Learning LAB

Week 04

Topics to cover

- KNN
- SVM
- Voting Classifier
- K-mean clustering

KNN Algorithm

- Implementation of KNN

KNN

import matplotlib.pyplot as plt

- from sklearn.datasets import load_wine
- from sklearn.model_selection import train_test_split
- from sklearn.neighbors import KNeighborsClassifier
- from sklearn.metrics import accuracy_score
-

KNN

Load the Wine dataset

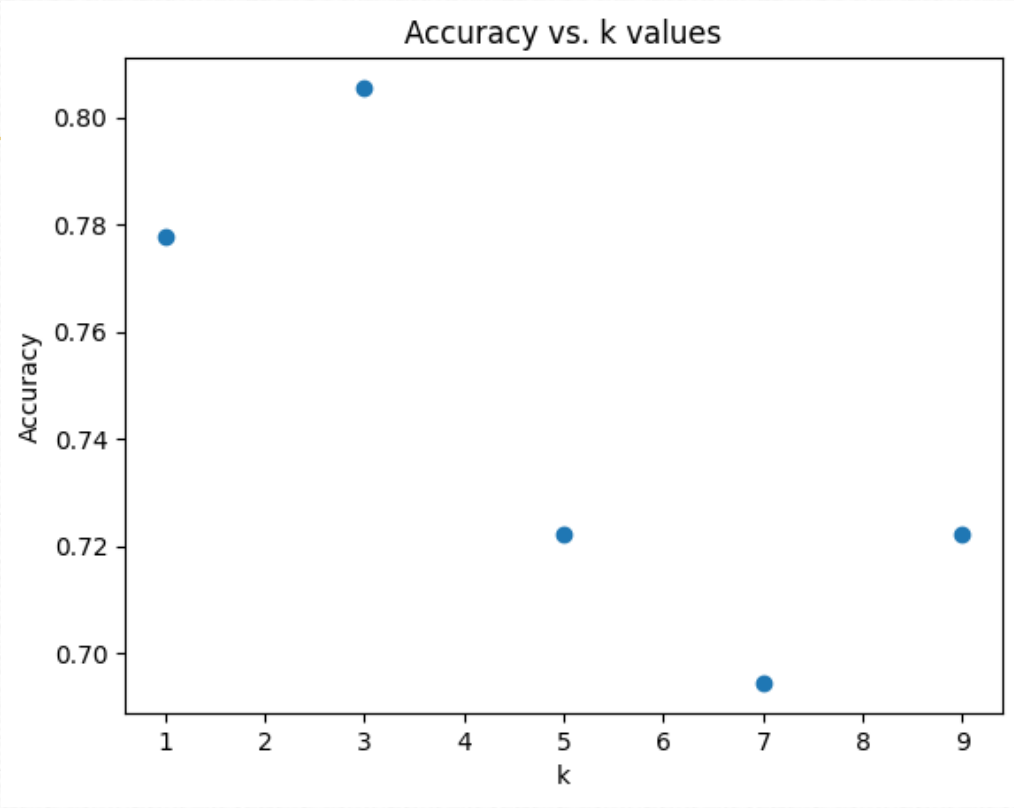
- wine = load_wine()
- X, y = wine.data, wine.target
- # Split the data into training and test sets
- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

KNN for different values

```
# Define the k values to test
k_values = [1, 3, 5, 7, 9]
# Train a KNN classifier for each value of k and record the accuracy scores
accuracy_scores = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    accuracy = knn.score(X_test, y_test)
    accuracy_scores.append(accuracy)
```

KNN for different values

- # Plot accuracy against k values
- plt.scatter(k_values, accuracy_scores)
- plt.xlabel("k")
- plt.ylabel("Accuracy")
- plt.title("Accuracy vs. k values")
- plt.show()



KNN for different values

```
k_values = range(1, 30)
accuracy_scores = []

# Apply KNN for different values of k
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracy_scores.append(accuracy)
# Print the best value of k
best_k = k_values[accuracy_scores.index(max(accuracy_scores))]
print("Best value of k:", best_k)
```

```
Best value of k: 3
```

Change p values

- import matplotlib.pyplot as plt
- from sklearn.datasets import load_wine
- from sklearn.model_selection import train_test_split
- from sklearn.neighbors import KNeighborsClassifier
- from sklearn.metrics import accuracy_score
  # Load the Wine dataset
- wine = load_wine()
- X, y = wine.data, wine.target
  # Split the data into training and test sets
- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Change p values

```
# Define the p values to test
p_values = [1, 2, 3, 4, 5]
# Train a KNN classifier for each value of p and record the accuracy scores
accuracy_scores = []
for p in p_values:
    knn = KNeighborsClassifier(n_neighbors=5, p=p)  # Adjust n_neighbors as needed
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracy_scores.append(accuracy)
```

Change p values

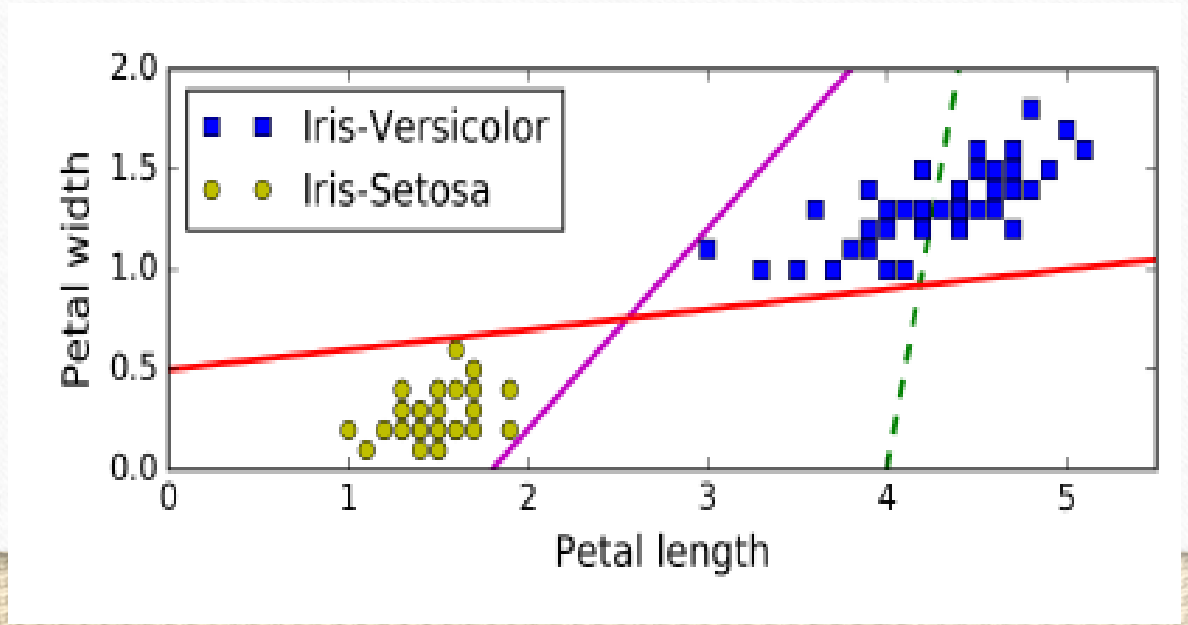
Print the accuracy scores for each p value

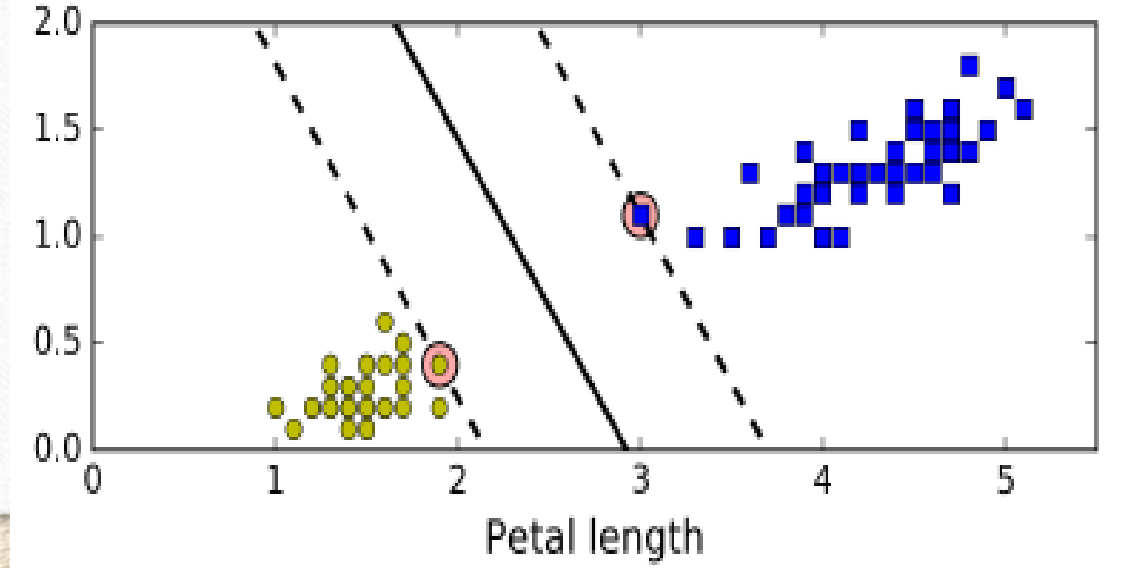
- print("Accuracy scores for each p value:")
- for p, accuracy in zip(p_values, accuracy_scores):
- print(f"p={p}: {accuracy}")
- print()

```
Accuracy scores for each p value:
p=1: 0.8055555555555556
p=2: 0.7222222222222222
p=3: 0.7222222222222222
p=4: 0.7222222222222222
p=5: 0.7222222222222222
```

SVM

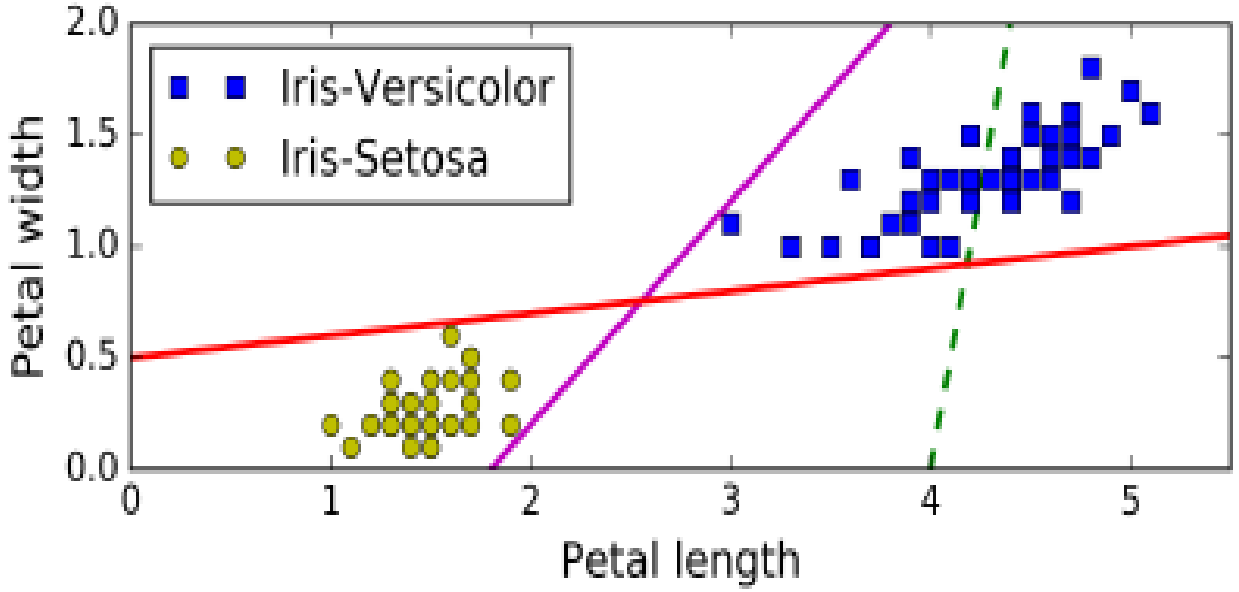
- A *Support Vector Machine* (SVM) is a very powerful and versatile Machine Learning model, capable of performing linear or nonlinear classification, regression, and even outlier detection
- You can think of an SVM classifier as fitting the widest possible street (represented by the parallel dashed lines) between the classes.
- This is called *large margin classification*

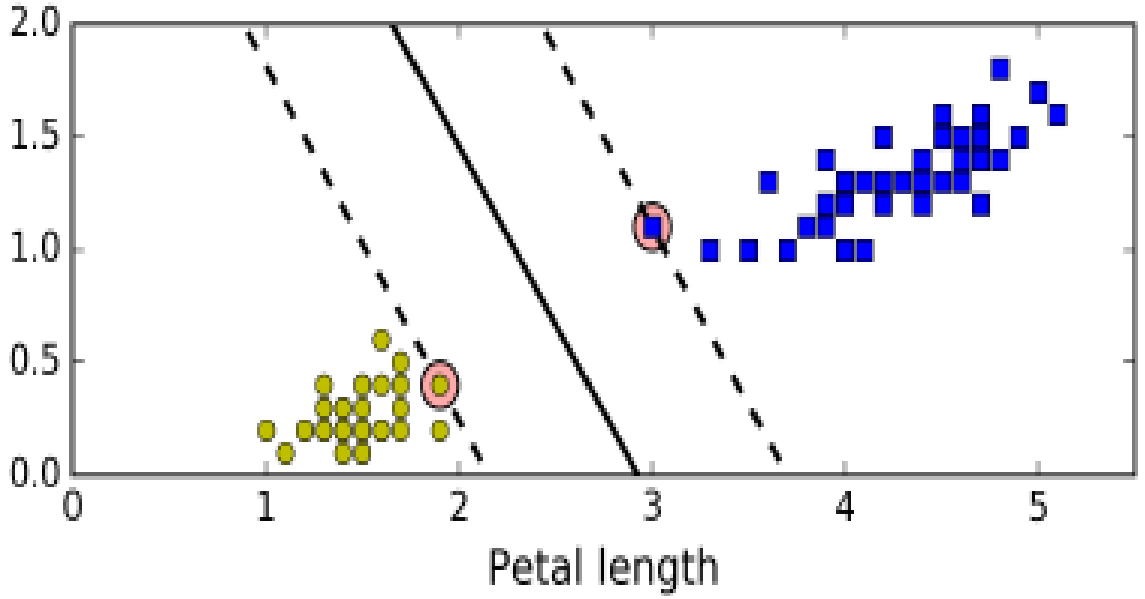




SVM

- Notice that adding more training instances "off the street" will not affect the decision boundary at all: it is fully determined (or "supported") by the instances located on the
- edge of the street. These instances are called the *support vectors*





SVM

```
# Import all relevant libraries

from sklearn.svm import SVC

import numpy as np

import pandas as pd

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn import preprocessing
from sklearn.model_selection import train_test_split, cross_val_score
import warnings
warnings.filterwarnings("ignore")
```

SVM

```
df = pd.read_csv('/content/income_evaluation.csv')

df.head()
```

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

SVM

```
# Identify categorical and numerical features
categorical_features = df.select_dtypes(include=['object']).columns
numerical_features = df.select_dtypes(exclude=['object']).columns


# Apply one-hot encoding to categorical features
df = pd.get_dummies(df, columns=categorical_features, drop_first=True)


# Separate features and target variable again after one-hot encoding
X = df.drop("income_ >50K", axis=1)  # Adjust target column name based on
get_dummies output
y = df["income_ >50K"]


# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=101)


# Scale numerical features
scaler = StandardScaler()
X_train[numerical_features] = scaler.fit_transform(X_train[numerical_features])
X_test[numerical_features] = scaler.transform(X_test[numerical_features])


display(X_train.head())
```

	age	fnlwgt	education-num	capital-gain	capital-loss	hours-per-week	workclass_ Federal-gov	workclass_ Local-gov	workclass_ Private	workclass_ Self-emp-inc	...	native-country_ Portugal	native-country_ Puerto-Rico	native-country_ Scotland	native-country_ South	native-country_ Puerto-Rico	native-country_ Scotland	native-country_ South	native-country_ Taiwan	native-country_ Thailand	native-country_ Trinadad&Tobago	native-country_ United-States	native-country_ Vietnam	native-country_ Yugoslavia
4400	-0.996110	-0.550200	-1.222152	-0.147881	-0.221529	-2.351924	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False	True	False	False
2598	0.181302	-0.077999	-0.427479	-0.147881	-0.221529	0.791095	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False	True	False	False
496	-1.364051	0.666475	-0.030143	-0.147881	-0.221529	-0.036015	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False	True	False	False
4915	-1.511228	-0.028649	-1.222152	-0.147881	-0.221529	-1.690236	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False	True	False	False
1341	1.358714	-0.955980	1.559203	-0.147881	-0.221529	-0.036015	False	False	True	False	...	False	False	False	False	False	False	False	False	False	False	True	False	False

5 rows × 97 columns

SVM

```
svc = SVC(random_state=101)
svc.fit(X_train,y_train)

accuracies = cross_val_score(svc,X_train,y_train,cv=5)

print("Train Score:", np.mean(accuracies))

print("Test Score:", svc.score(X_test,y_test))
```

```
Train Score: 0.8461355529131985
Test Score: 0.8488593155893536
```

Tunning

```
from sklearn.model_selection import GridSearchCV

grid = {
    'C':[0.01,0.1,1,10],
    'kernel' : ["linear","poly","rbf","sigmoid"],
    'degree' : [1,3,5,7],
    'gamma' : [0.01,1]
}

svm  = SVC ()

svm_cv = GridSearchCV(svm, grid, cv = 5)

svm_cv.fit(X_train,y_train)

print("Best Parameters:",svm_cv.best_params_)

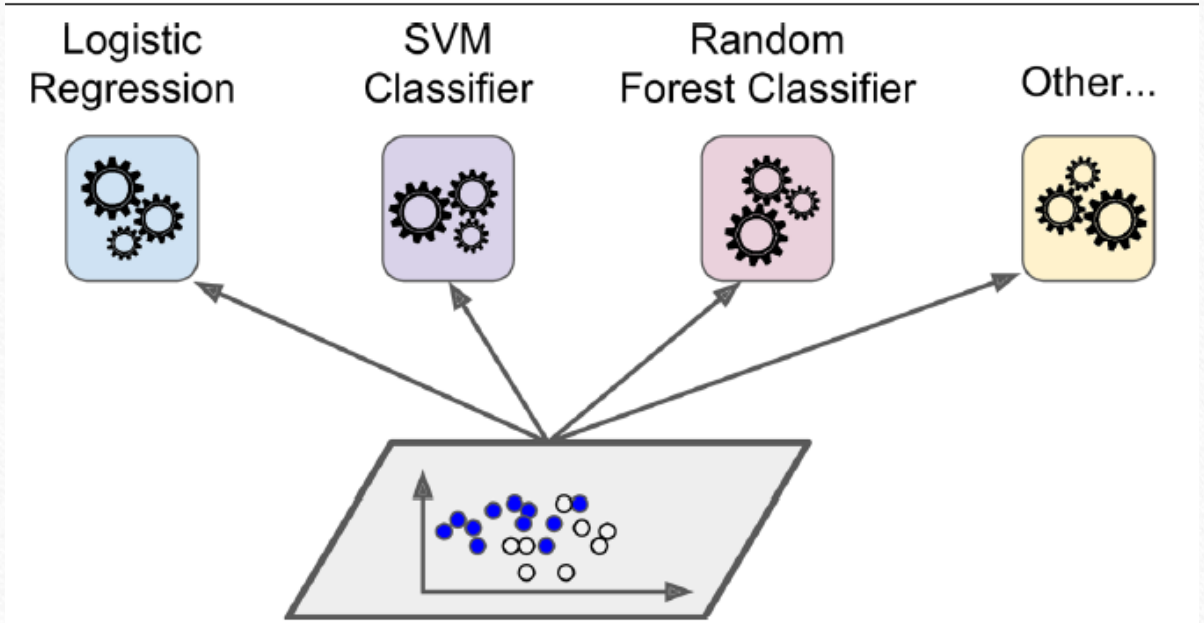
print("Train Score:",svm_cv.best_score_)

print("Test Score:",svm_cv.score(X_test,y_test))
```

```
Best Parameters: {'C': 10, 'degree': 1, 'gamma': 0.01, 'kernel': 'rbf'}
Train Score: 0.8482758620689654
Test Score: 0.8488593155893536
```

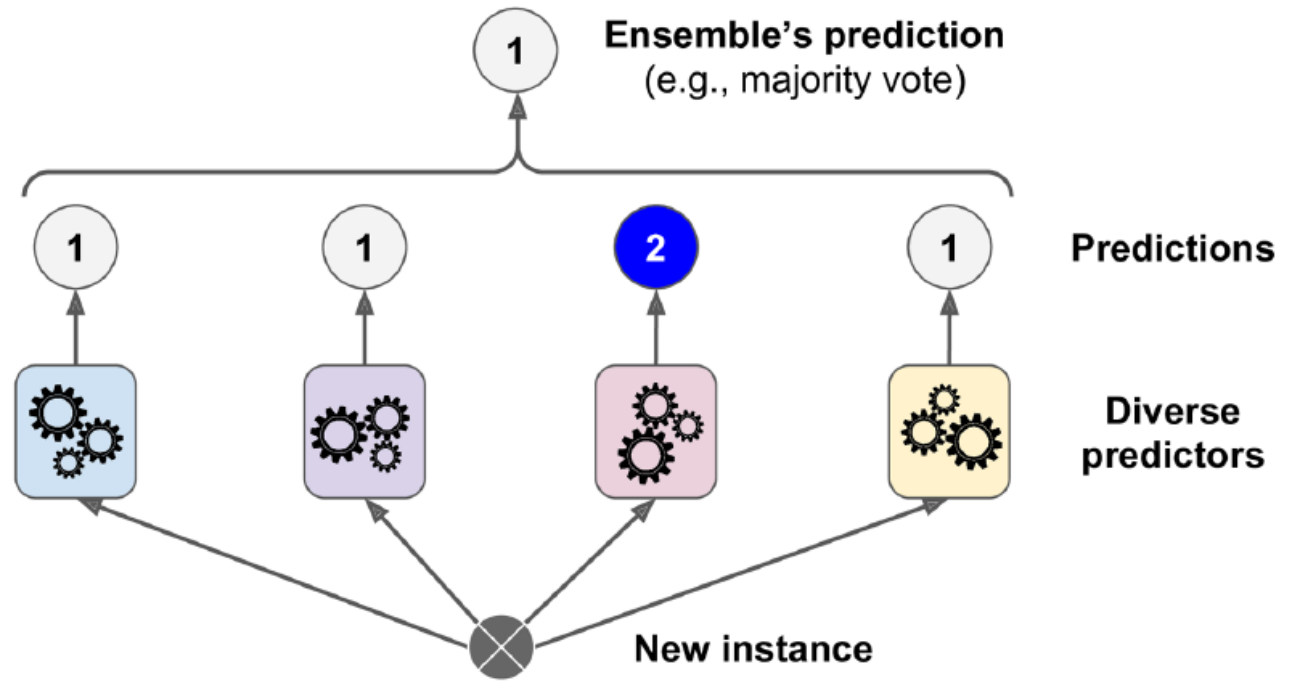
Voting Classifiers

- Suppose you have trained a few classifiers, each one achieving about 80% accuracy. You may have a Logistic Regression classifier, an SVM classifier, a Random Forest classifier, a K-Nearest Neighbors classifier, and perhaps a few more



Voting Classifiers

- A very simple way to create an even better classifier is to aggregate the predictions of each classifier and predict the class that gets the most votes. This majority-vote classifier is called a *hard voting* classifier



Voting Classifiers

- from sklearn.ensemble import RandomForestClassifier
- from sklearn.ensemble import VotingClassifier
- from sklearn.linear_model import LogisticRegression
- from sklearn.svm import SVC
- log_clf = LogisticRegression()
- rnd_clf = RandomForestClassifier()
- svm_clf = SVC()
- voting_clf = VotingClassifier(
- estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)], voting='hard')
- voting_clf.fit(X_train, y_train)
- print(voting_clf.predict([some_digit]))

Voting Classifiers

- from sklearn.metrics import accuracy_score
- for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
- clf.fit(X_train, y_train)
- y_pred = clf.predict(X_test)
- print(clf.__class__.__name__, accuracy_score(y_test, y_pred))

```
LogisticRegression 0.9195171026156942
RandomForestClassifier 0.9255533199195171
SVC 0.9537223340040242
```

K-Means Clustering

- 1 - Imports
- 2 - Loading the data
- 3 - Exploratory Analysis
- 4 - Preprocessing the data
- 5 - Trainin K-Means Clustering to Find Clusters
- 6 - Evaluating K-Means Clustering

IMPORTS AND DATA LOADING

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt


# Step 1: Load the dataset
data = pd.read_csv("/content/mushroomss.csv")


# Display first few rows
print("Dataset Preview:")
print(data.head())


# Step 2: Check for missing values
print("\nMissing Values in Each Column:")
print(data.isnull().sum())
```

EXPLORATORY DATA ANALYSIS

```
print(f"Dataset shape: {mushroom_data.shape}")
print(f"Target
distribution:\n{mushroom_data['class'].value_counts()}")
```

```
Dataset shape: (8124, 23)
Target distribution:
class
e    4208
p    3916
Name: count, dtype: int64
```

EXPLORATORY DATA ANALYSIS

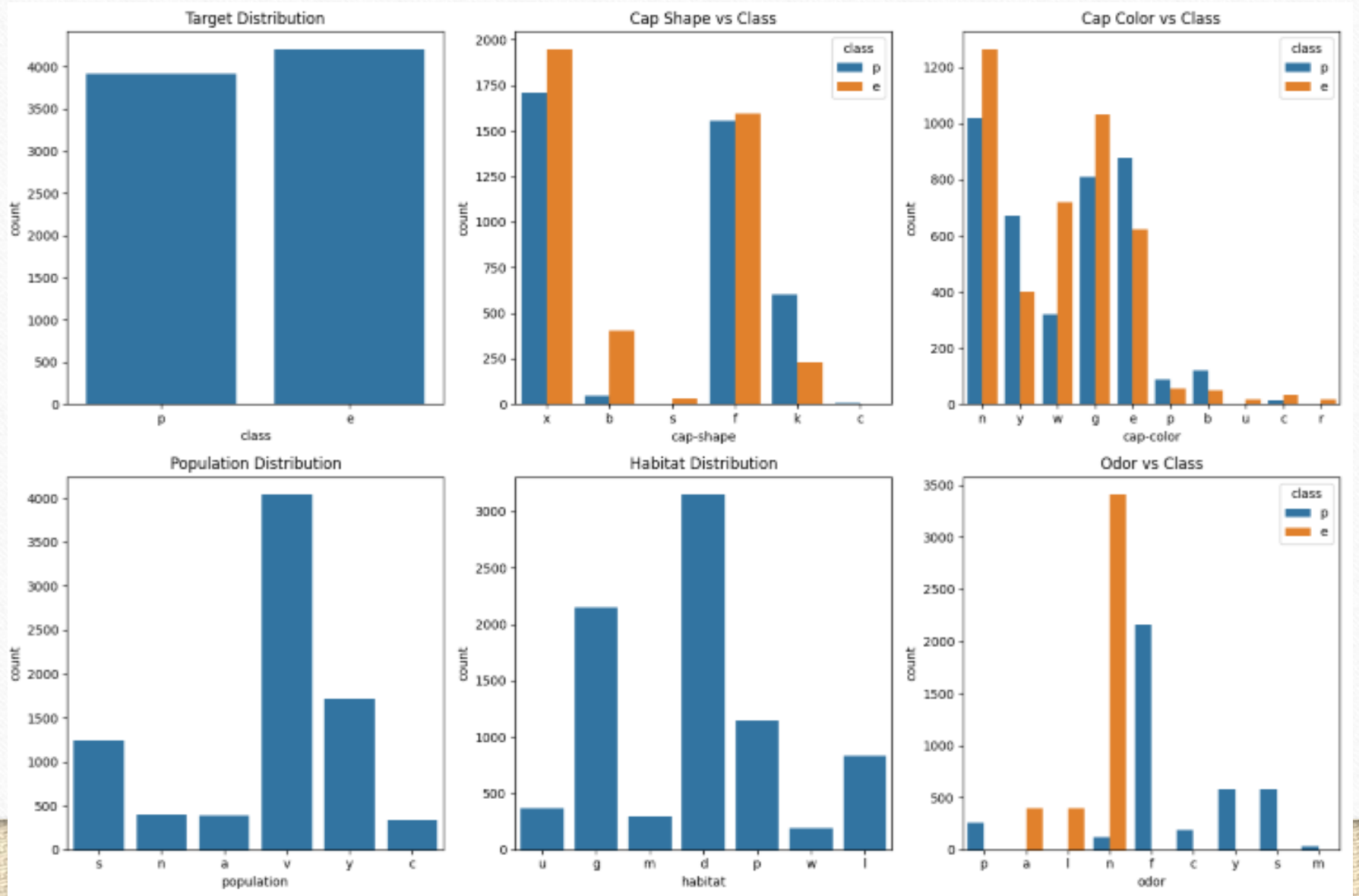
```
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
sns.countplot(data=mushroom_data, x='class')
plt.title('Target Distribution')

plt.subplot(2, 3, 2)
sns.countplot(data=mushroom_data, x='cap-shape', hue='class')
plt.title('Cap Shape vs Class’)
plt.subplot(2, 3, 3)
sns.countplot(data=mushroom_data, x='cap-color', hue='class')
plt.title('Cap Color vs Class')
```

EXPLORATORY DATA ANALYSIS

```
plt.subplot(2, 3, 4)
sns.countplot(data=mushroom_data, x='population')
plt.title('Population Distribution')
plt.subplot(2, 3, 5)
sns.countplot(data=mushroom_data, x='habitat')
plt.title('Habitat Distribution')
plt.subplot(2, 3, 6)
sns.countplot(data=mushroom_data, x='odor', hue='class')
plt.title('Odor vs Class')
plt.tight_layout()
plt.show()
```

EXPLORATORY DATA ANALYSIS



DATA PREPROCESSING

```
#  Handle missing values
mushroom_df = mushroom_data.dropna()

# Separate features and target
mushroom_features = mushroom_df.drop('class', axis=1)
mushroom_labels = mushroom_df['class']
# Encode categorical features
encoder = OrdinalEncoder()
mushroom_prepared =
    encoder.fit_transform(mushroom_features)
mushroom_prep_df = pd.DataFrame(mushroom_prepared,
columns=mushroom_features.columns)
```

MODEL IMPLEMENTATION

KMEANS CLUSTERING

```
# Create and fit KMeans model
k_clust = KMeans(n_clusters=2, random_state=42)

k_clust.fit(mushroom_prep_df)

# Get results
k_labels = k_clust.labels_
```

MODEL EVALUATION

```
 # Map true labels to numerical values
map_dict = {'e': 0, 'p': 1}
true_labels = mushroom_labels.map(map_dict)


# Calculate accuracy
accuracy = accuracy_score(true_labels, k_labels)
print(f"KMeans Accuracy: {accuracy:.4f}")
```

```
KMeans Accuracy: 0.7090
```

Confusion matrix

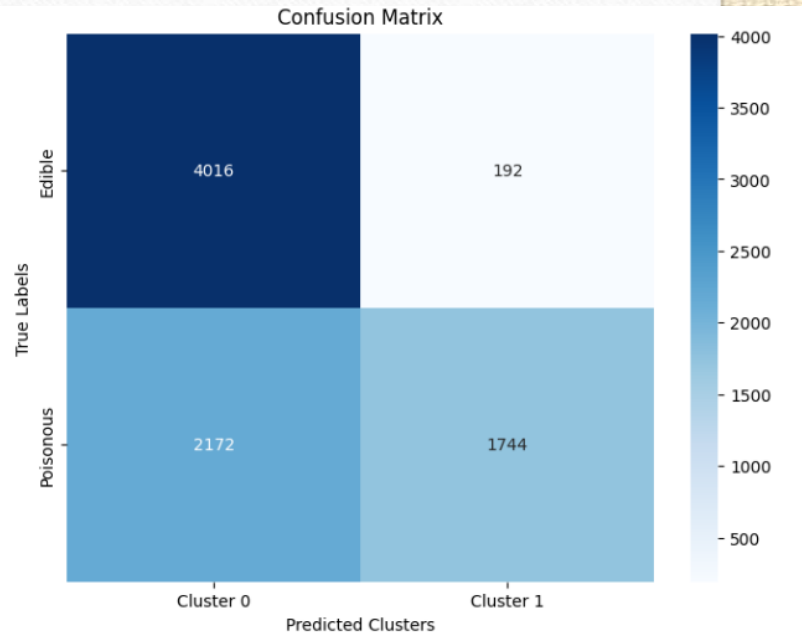
```
plt.figure(figsize=(8, 6))

cm = confusion_matrix(true_labels, k_labels)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Cluster 0', 'Cluster 1'],
            yticklabels=['Edible', 'Poisonous'])

plt.title('Confusion Matrix')
plt.xlabel('Predicted Clusters')
plt.ylabel('True Labels')
plt.show()

# Classification report
print("\nClassification Report:")
print(classification_report(true_labels, k_labels,
target_names=['Edible', 'Poisonous']))
```



Classification Report:

	precision	recall	f1-score	support
Edible	0.65	0.95	0.77	4208
Poisonous	0.90	0.45	0.60	3916
accuracy			0.71	8124
macro avg	0.77	0.70	0.68	8124
weighted avg	0.77	0.71	0.69	8124

UNSUPERVISED LEARNING

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt


# Step 1: Load the dataset
data = pd.read_csv("/content/mushroomss.csv")


# Display first few rows
print("Dataset Preview:")
print(data.head())


# Step 2: Check for missing values
print("\nMissing Values in Each Column:")
print(data.isnull().sum())


# Step 3: Encode categorical features (if any)
label_encoders = {}
for column in data.columns:
    if data[column].dtype == 'object':
        le = LabelEncoder()
        data[column] = le.fit_transform(data[column])
        label_encoders[column] = le
```

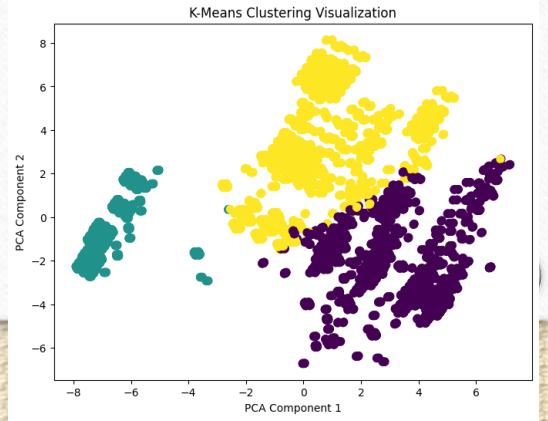
UNSUPERVISED LEARNING

```
# Step 4: Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
data['Cluster'] = kmeans.fit_predict(data)


# Step 5: Display cluster centers and counts
print("\nCluster Centers:\n", kmeans.cluster_centers_)
print("\nCluster Counts:\n", data['Cluster'].value_counts())


# Step 6: Visualize clusters using PCA (2D)
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(data.drop('Cluster', axis=1))
plt.figure(figsize=(8,6))
plt.scatter(reduced_data[:,0], reduced_data[:,1],
c=data['Cluster'], cmap='viridis', s=50)
plt.title('K-Means Clustering Visualization')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```

```
Cluster Counts:
 Cluster
0    3535
2    2707
1    1882
```



Activity 1

KNN for Credit Card Fraud Detection

Data Set: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?resource=download

Use KNN to detect fraudulent transactions in a highly **imbalanced dataset**, where frauds make up less than 0.2% of all samples.

1. Load the dataset and split it into features X and target y.
2. Apply **SMOTE (Synthetic Minority Oversampling Technique)** or **ADASYN** to handle class imbalance.
3. Scale features using **StandardScaler**.
4. Train a **KNN classifier** with k=5, then tune k using **GridSearchCV** for best results.
5. Compare model performance using:
    1. Accuracy
    2. F1-Score

Activity 2

SVM for Brain Tumor Detection

Data Set: https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection
Load the dataset and preprocess all MRI images:
Convert to grayscale
Resize to 64×64
Flatten each image
Split data into training and testing sets (80:20).
Normalize features using StandardScaler().
Train an **SVM (RBF kernel)** with C=10 and gamma='scale'.
Evaluate performance using accuracy, classification report, and confusion matrix.
Display a few test images with predicted and actual labels.

Activity 3

Dataset:
`Mall_Customers`
Tasks:
1.Load the dataset and display it.
2.Apply **K Mean** & **DBSCAN Algorithm** and plot clusters using *Annual Income* vs *Spending Score*.
3.Scale features using **StandardScaler** and re-run K-Means.
4.Compare both results did scaling change clusters?
5.Use the **Elbow Method** to find the best `k`.
6.Briefly interpret each cluster.
Also:
Add a fake feature `"LuckyNumber"` (random 0–100) and explain its effect on clustering.